

FILMORY

Alumnas: Marina Makarova | Melina Higa
Carrera: Diseño y Programación Web
Comisión: DWN3AV
Año: Tercer cuatrimestre - 2026

INTRODUCCIÓN

La aplicación web **Filmory** fue pensada con el propósito de ofrecer una plataforma centralizada y eficiente para organizar las películas que el usuario consume.

En la actualidad, **el público consume contenido multimedia** a través de múltiples plataformas de streaming descentralizadas, lo que genera la necesidad de contar con un espacio único para planificar, registrar y opinar sobre dichos contenidos.

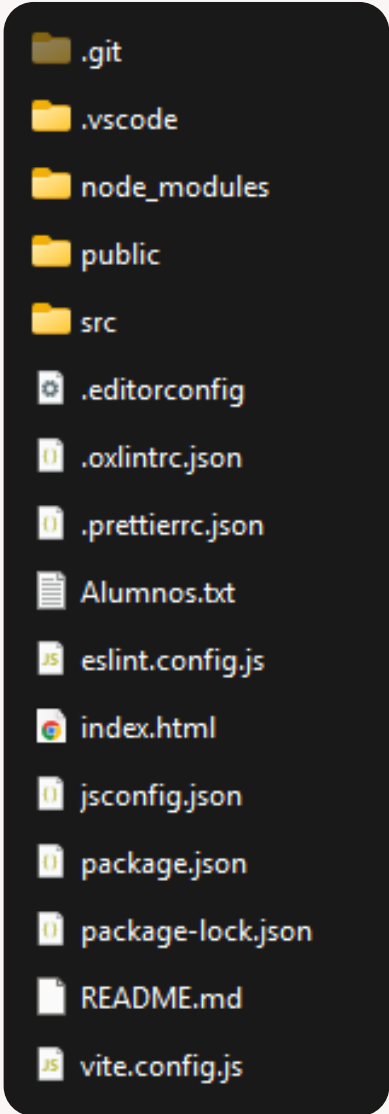
El público objetivo abarca a cinéfilos, estudiantes de cine y espectadores casuales que buscan mantener un control ordenamiento de sus listas de reproducción mediante tres estados clave: películas por ver, películas en curso y películas ya visualizadas.



CONFIGURACIÓN DEL ENTORNO

Se implementaron las siguientes herramientas y configuraciones técnicas:

- **Vite:** Utilizado como entorno de desarrollo y empaquetador para la creación de componentes de archivo único bajo la versión 3 de Vue.
- **Vue Router:** Configurado en la raíz del proyecto para gestionar el enrutamiento de manera dinámica y fluida.
- **ESLint y Prettier:** Incorporados y sincronizados mediante archivos de configuración (eslint.config.js, .prettierrc.json y .editorconfig) para automatizar el formateo del código, prevenir malas prácticas y garantizar la ausencia de errores o advertencias en la consola de desarrollo.



VISTAS Y FLUJO DE NAVEGACIÓN

La arquitectura del sitio se compone de un máximo de **tres vistas esenciales** dentro del flujo general comunicadas a través de Vue-Router:

1. **HomeView (Inicio):** Funciona como pantalla de bienvenida e introducción a la plataforma. Detalla los motivos de la elección de la temática y provee un acceso directo intuitivo hacia las demás secciones del sitio.
2. **AgregarView (Agregar Película):** Aloja el componente del formulario de carga. Su rol es puramente operativo, permitiendo la inserción de nuevos elementos de forma aislada y limpia dentro de la interfaz.
3. **BibliotecaView (Mi Biblioteca):** Representa el núcleo visual de la aplicación, donde se listan las películas registradas segmentadas en tres columnas correspondientes a sus estados de visualización.



1

VISTA DEL INICIO



Es la primera vista que ve el usuario al entrar a la aplicación. Su rol funcional es **introdutorio y contextual**: presenta formalmente el nombre del proyecto, expone los fundamentos teóricos que motivaron la elección de la temática y provee al usuario un flujo claro de navegación hacia las áreas operativas mediante botones de acción directa.

Funcionalidades de Vue y su relación con el contenido:

- **Enrutamiento dinámico:** Se implementa la etiqueta `<RouterLink>` vinculada a las rutas de 'agregar' y 'bibliotec'a. Esto permite cambiar de vista instantáneamente sin recargar el navegador.
- **Imágenes dinámicas:** Se utiliza el enlazado de atributos (`:src="heroCinema"`) referenciando la imagen importada desde los assets locales, permitiendo que Vite resuelva correctamente la ruta en el empaquetado final.

2

VISTA DEL FORMULARIO



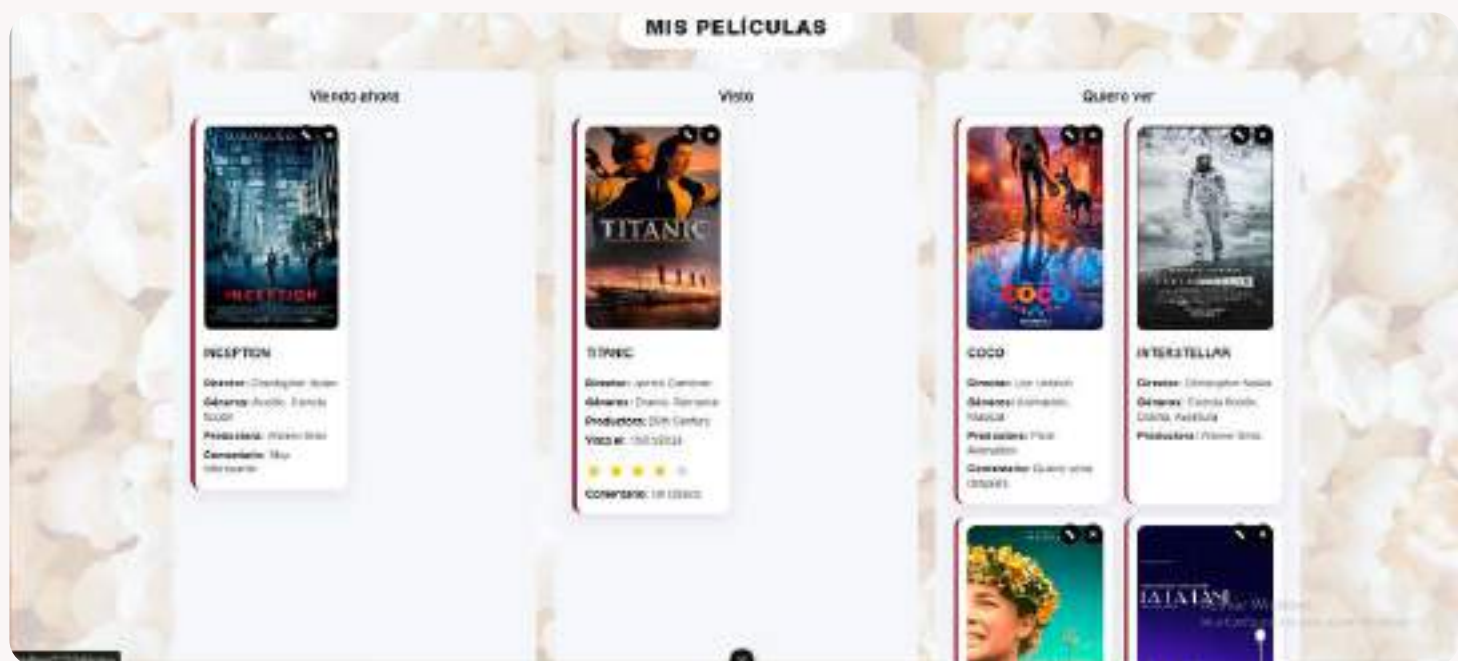
Esta vista **centraliza el alta de nuevos registros de películas**. Su función es ofrecer un **formulario interactivo** estructurado para capturar los metadatos de una película y las variables condicionales asociadas a su visualización.

Funcionalidades de Vue y su relación con el contenido:

- **Vinculación bidireccional:** Se utiliza *v-model.trim* en los campos de texto para limpiar espacios en blanco. Los *checkboxes* de géneros y los *<select>* están enlazados al objeto reactivo *formData*.
- **Estructuras condicionales (v-if):** Si *formData.estado* es estrictamente igual a 'visto', Vue monta dinámicamente en el DOM los campos adicionales de 'fecha' y 'calificación por estrellas'.
- **Renderizado de listas (v-for):** Las opciones de las 'productoras' y los 'estados' se construyen iterando arrays de objetos estáticos declarados en el *data()*, optimizando el template HTML.

3

VISTA DE LA BIBLIOTECA



Su función es **organizar, listar, modificar y dar de baja las películas almacenadas**. El contenido se divide visualmente en tres columnas organizativas basadas en el estado del consumo audiovisual.

Funcionalidades de Vue y su relación con el contenido:

- **Filtrado reactivo:** La vista utiliza el método *peliculasPorEstado(estado)* en un bucle *v-for*. Esto filtra el contenido del array general, distribuyendo automáticamente cada tarjeta *PeliculaCard* en la columna correspondiente.
- **Manejo de eventos nativos (@drag):** Las tarjetas y las columnas utilizan manejadores de eventos nativos de Vue (*@dragstart*, *@dragover.prevent*, *@drop*) para procesar el arrastre y soltado de elementos, cambiando el estado reactivo de la película al vuelo.
- **Clases dinámicas (:class):** Si una película recibe una puntuación máxima, su componente adopta dinámicamente una clase especial (*pelicula-favorita*).
- **Modo edición condicional:** Un *v-if="editando"* alterna la visualización estática de la tarjeta por un *sub-formulario* de edición con campos vinculados a *datosEdicion*, permitiendo editar el objeto y mandar los cambios a la vista padre.

COMPONENTES REUTILIZABLES

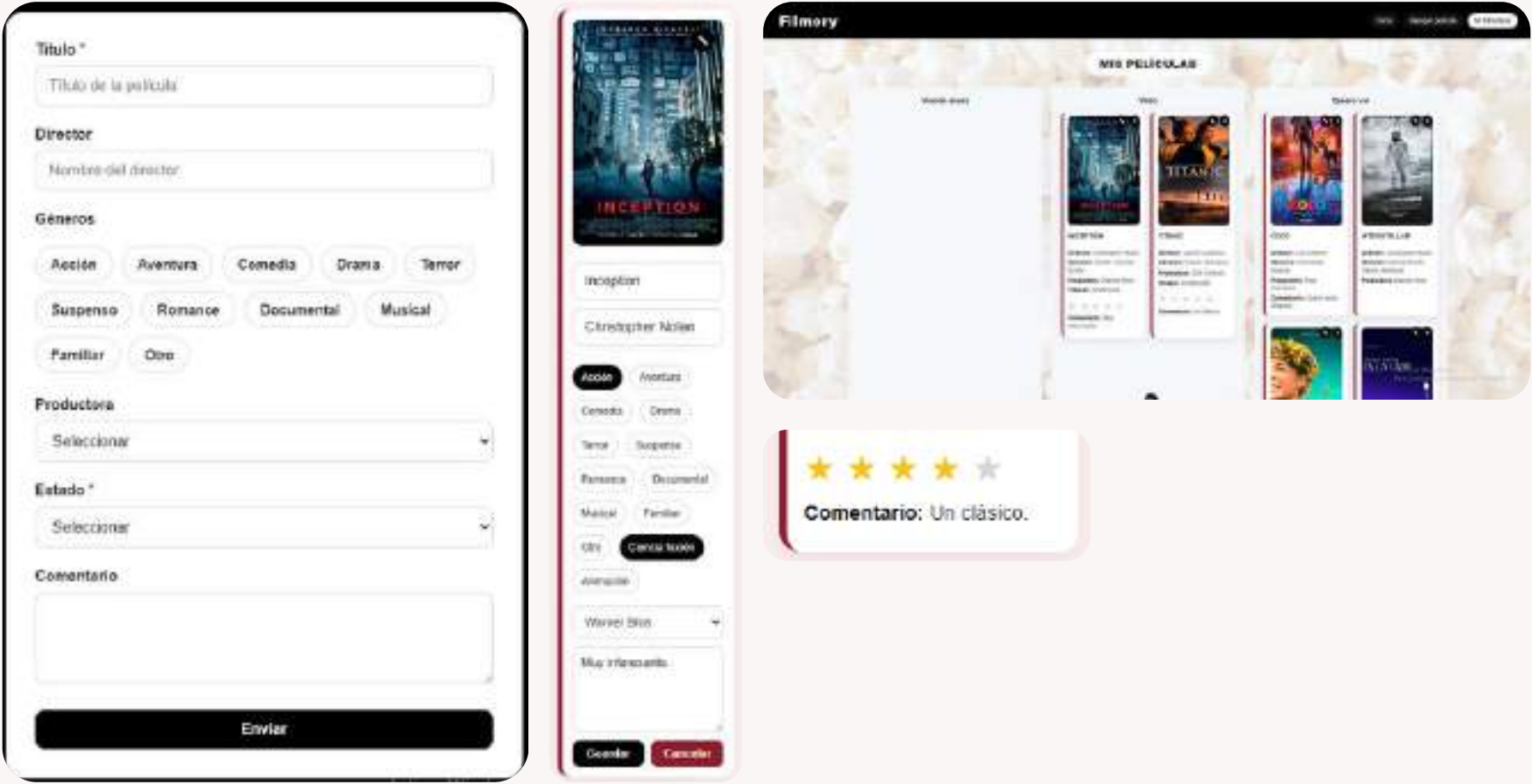
Se desarrollaron **dos componentes específicos** con lógica y responsabilidades bien delimitadas dentro de la carpeta */components*:

- **PeliculaForm.vue:** Gestiona la recolección de datos mediante la directiva *v-model*. Cuenta con métodos de validación interactiva interna y tras comprobar que los campos obligatorios estén completos, procesa la estructura del objeto y la despacha al componente padre mediante la instrucción *\$emit('guardar-pelicula', nuevaPelicula)*.
- **PeliculaCard.vue:** Es encargado de la representación gráfica individual de cada película dentro de la biblioteca. Cuenta con una lógica interactiva propia para el manejo inline del modo edición y la acción interactiva de la puntuación por estrella, comunicando las actualizaciones a través de eventos.

DATOS ABM EN LOCALSTORAGE

El ciclo completo de Altas, Bajas y Modificaciones (ABM) se administra de manera local y persistente utilizando *LocalStorage*:

- **Alta (Create):** Se ejecuta en *AgregarView.vue* al recibir el evento del formulario, empujando el nuevo registro al array de películas y sobrescribiendo la clave *filmory_peliculas* en el espacio local.
- **Baja (Delete):** Implementada en *BibliotecaView.vue* a través del método *eliminarPelicula(id)*, el cual solicita una confirmación explícita al usuario antes de remover el elemento del vector y actualizar el almacenamiento.
- **Modificación (Update):** Disponible de dos formas interactivas: de forma directa a través del modo edición de cada tarjeta (*PeliculaCard.vue*), y de manera dinámica mediante eventos de arrastre (*Drag and Drop*), permitiendo al usuario mover físicamente las tarjetas entre las columnas de estados (visto, viendo, quiero), actualizando los metadatos de fechas y persistiendo los cambios de manera inmediata.



VALIDACIONES DEL FORMULARIO, MENSAJES DE ERROR Y ANIMACIONES

Para asegurar la integridad de los datos de la aplicación sin recurrir a las validaciones genéricas por defecto de los navegadores, se aplicó el atributo *novalidate* en la etiqueta de formulario.

La validación se ejecuta en el método interactivo *validarFormulario()*. Evalúa de forma estricta las siguientes condiciones:

- 1. Que el campo de texto destinado al ‘Título’ no se encuentre vacío.
- 2. Que se haya seleccionado de forma obligatoria un ‘Estado’ inicial válido para la película dentro del menú desplegable.

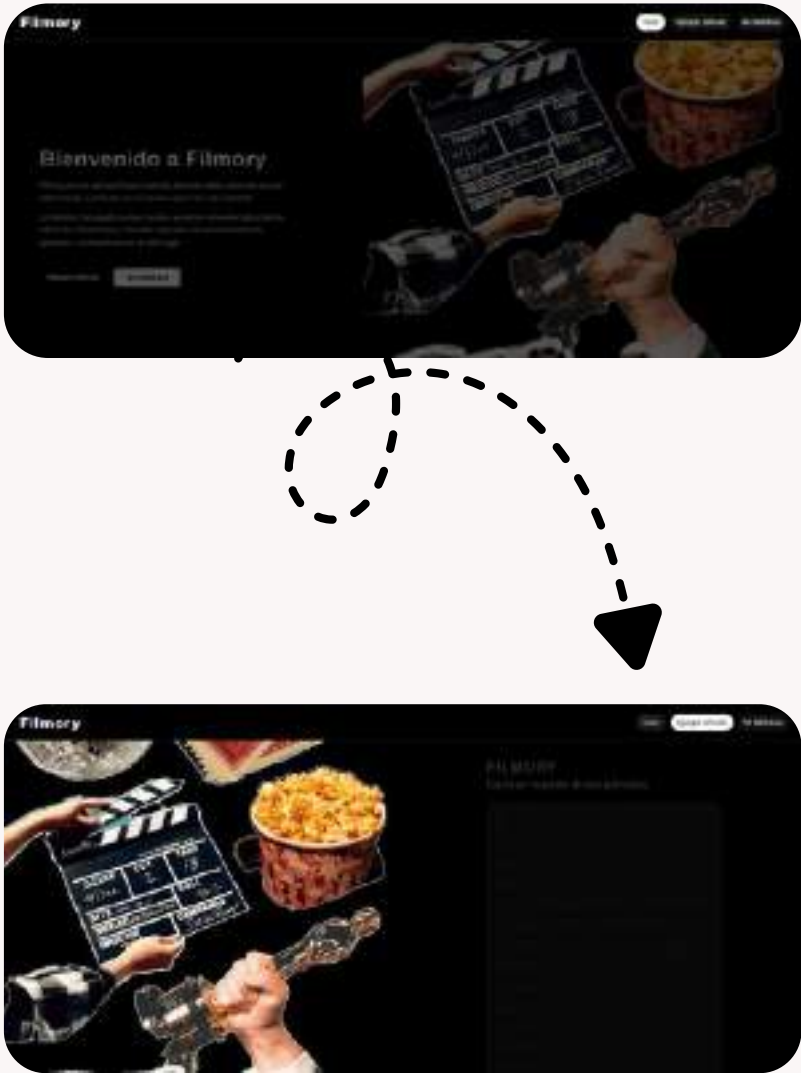
En caso de no superar estas condiciones, se bloquea el envío y se renderizan dinámicamente mensajes de error claros y personalizados en la zona superior de la pantalla mediante estructuras condicionales *v-if* iterando sobre un array de errores.



Finalmente, se incorporó la librería externa **Animate.css** a nivel global mediante su importación en el archivo de entrada del sistema (main.js).

Para **enriquecer la experiencia de usuario y suavizar las transiciones de la interfaz** en relación con la presentación del contenido. Principalmente se añadió *animate__animated animate__fadeIn*:

- En el contenedor principal de la pantalla de bienvenida (*HomeView.vue*), permitiendo una aparición fluida del texto introductorio.
- En el bloque de carga de formularios (*AgregarView.vue* y *PeliculaForm.vue*), suavizando el ingreso a la vista interactiva.



MUCHAS GRACIAS